

Latent Teleportation

Forecasting Diffusion Trajectories for Faster Image Generation
Schedule-Aware Forecasts and Retrieved Trajectory Priors

Lee Penkman

CuteDSL / OmniServe

leepenkman@gmail.com

August 27, 2026

Abstract

Latent teleportation accelerates diffusion by forecasting a later refinement state and resuming the denoiser from that state. We study three complementary signals for the forecast: the query trajectory’s local momentum, non-uniform anchor placement, and corrections retrieved from similar historical trajectories. The resulting view is a schedule-aware manifold warp: preserve the query’s spatial path, estimate how far and where it should move, and spend full denoiser evaluations at the anchors where correction matters most.

On 200 captured Z-Image trajectories, one schedule-calibration scalar reduces five-fold held-out forecast error by 20–37% over first-order Taylor for horizons $k = 1 \dots 8$. Aligning four forecast intervals reduces local error to $\text{relL2} = 0.215$, 35.3% below uniform Taylor, while four distinct near-optimal schedules remain within 2.4% of the best. We also evaluate retrieval architectures that use prompt similarity, observed latent motion, candidate pruning, and a learned local-neighbour residual gate. This makes historical trajectories useful as a small correction to the query’s own forecast rather than as copied latent motion. The best eight-neighbour residual lowers held-out error by 3.3%, improves a six-prompt decoded long-warp comparison by 0.36 dB PSNR and 0.015 SSIM, and costs 0.80 ms/query in a resident single-thread CPU benchmark versus 5.84 ms for dense trajectory attention. Variance-aware shrinkage raises the decoded comparison to 0.7883 SSIM. A 7,312-parameter focused adapter improves the scalar cache gate at every evaluated horizon, reaches $\text{relL2} = 0.223$, and raises decoded PSNR to 21.92 dB; its 31 KB weights are published with schedule coefficients and raw metrics.

We separately test an exact serving shortcut for repeated prompt/seed/configuration keys. On a live shared RTX 5090 native service, CPU-resident latent replay is pixel-identical on all fresh-key promotion cases and gives a paired median prime-to-replay speedup of $1.491\times$ when resuming at the final scheduled step. A four-hit soak completes without out-of-memory failure and post-request GPU allocation plateaus. This exact repeated-request tier is distinct from the approximate forecasters used for unseen prompts.

The paper connects these mechanisms to Align Your Steps, consistency models, latent consistency models, flow matching, reflow, and feature-reuse methods; shows how lane-specific anchors can provide varied batch outputs with compacted GPU work; and reports decoded-image grids, forecast quality, predictor overhead, and end-to-end serving measurements. The strongest production direction is to combine aligned anchors, calibrated local momentum, pruned retrieval, fused denoiser kernels, and memory-aware active-lane scheduling for long high-quality samplers.

1 Introduction

A diffusion sampler spends nearly all of its arithmetic evaluating one large denoiser many times over. The obvious economy is to evaluate it fewer times, and the literature offers two broad routes: change the model so that fewer steps suffice — distillation [Salimans and Ho, 2022],

consistency models [Song et al., 2023, Luo et al., 2023], adversarial distillation [Sauer et al., 2023] — or leave the model alone and reuse computation across steps [Ma et al., 2024, Selvaraju et al., 2024, Liu et al., 2025]. The second route is attractive operationally because it does not require retraining a production checkpoint.

Speculative decoding [Leviathan et al., 2023, Chen et al., 2023] suggests a third framing. In language models, the win comes not from a better model but from a hardware fact: autoregressive decode is memory-bandwidth-bound, so a batch of $k + 1$ candidate tokens costs about what one costs, and a cheap drafter can propose tokens that the real model merely *verifies*. The appeal is that verification is exact — the accepted output is distributed exactly as the target model’s — so speculation is a pure latency optimisation rather than a quality trade.

Diffusion sampling looks superficially similar. It is iterative, each iteration is one pass over a transformer, and the sequence of latents $x_0, x_1, \dots, x_N$ is a smooth curve rather than a sequence of discrete symbols — if anything, easier to extrapolate than text. This paper asks which parts of the analogy carry, which do not, and what must change for the forecastability signal to become useful serving latency.

What latent teleportation is — and is not

The name describes a family of approximate trajectory shortcuts: estimate a local tangent or transport map, then warp a latent along the learned manifold to a later refinement state. It is not one fixed algorithm and not an exact analogue of language-model speculative decoding.

What it is	What it is not
A forecast, manifold warp, or warm start in a continuous diffusion latent trajectory.	An exact verifier or a distribution-preserving decoding method.
A family that can use momentum, aligned schedules, learned predictors, confidence gates, or retrieved trajectories.	A claim that every listed mechanism has already passed image and latency evaluation.
A controllable trade between denoiser calls and same-seed image retention.	A guarantee of end-to-end speedup when text, VAE, transfer, or scheduling costs dominate.
A model-serving co-design problem whose useful operating point can change with resolution, schedule, and batching.	A single checkpoint-specific result that should be extrapolated to every sampler or deployment.

Contributions.

1. **A predictability study** of the Z-Image [Tongyi Lab, Alibaba Group, 2025] latent trajectory: a (t, k) grid of forecast error for identity, first-order Taylor and scalar-affine predictors, establishing that the trajectory is genuinely forecastable and quantifying where (Section 5.1).
2. **A five-fold forecaster ablation** showing that one fitted schedule scalar improves over raw Taylor by 20–37% through $k = 8$, while an extra historical delta adds little and an unfitted second-order rule is worse (Section 5.3).
3. **A call-budget schedule ablation** connecting trajectory forecasting to Align Your Steps: four cross-validated forecast intervals improve local forecast error by 35.3%, while four staggered near-optimal schedules remain within 2.4% of the best (Section 5.4).
4. **A retrieval-conditioned forecaster study** comparing direct neighbour transport, prompt-only top- k , motion-agreement pruning, scalar and channel transport, and learned residual fusion under five-fold held-out evaluation (Section 5.5).

5. **Variance-aware and learned cache procedures:** an uncertainty shrinkage rule and a zero-initialized 7,312-parameter spatial adapter, evaluated by horizon and decoded in the full procedure grid. We publish the adapter weights and schedule table.
6. **An operating-point bottleneck diagnosis:** call reductions of $3.2\times$ yield $1.04\times$ wall-clock, and a skip-everything ablation caps at $1.10\times$, localising the bottleneck outside the denoiser (Section 5.6).
7. **Native serving ablations** covering exact repeated-request latent replay, GGUF quantisation, step caching, a multi-hit memory soak, and an sm_120 fused residual-RMS epilogue. We also replace an accidental four-billion-parameter dense latent combiner with a bounded-rank auxiliary. We distinguish pixel-exact serving results from approximate image retention and isolated kernel timing (Section 5.7).
8. **Image-level ablations** over learned speculation, Taylor forecasting, a destructive skip control, three draft lengths, and six prompt/style slices. We publish all panels as contact sheets with PSNR, SSIM, and source-file hashes (Section 5.8).
9. **An extension sketch to LoRA armies**, where adapters induce disjoint latent statistics and a shared trajectory cache needs an embedding normalisation network (Section 3.5).

2 Background and Related Work

Diffusion and flow samplers. We work with a rectified-flow / flow-matching formulation [Liu et al., 2023, Lipman et al., 2023] as used by Z-Image, sampled on a fixed schedule of N timesteps in a latent space produced by a VAE [Rombach et al., 2022], with a DiT-style denoiser [Peebles and Xie, 2023]. Writing Φ for one scheduler step,

$$x_{i+1} = \Phi(x_i, t_i, c), \tag{1}$$

with c the text conditioning. The cost of sampling is N evaluations of the denoiser inside Φ , plus one text encode and one VAE decode.

Step reduction by retraining. Progressive distillation [Salimans and Ho, 2022], consistency models [Song et al., 2023], latent consistency models [Luo et al., 2023] and adversarial distillation [Sauer et al., 2023] all produce a new checkpoint that needs fewer steps. LCMs explicitly target high-resolution generation in two to four steps; this is powerful but consumes the very redundancy that a post-training skip method would otherwise exploit. The combination is therefore role-dependent: a consistency model can serve as a cheap endpoint proposer or training target, while teleportation can target a longer high-quality teacher schedule. Z-Image Turbo is already distilled, which matters for interpreting our results (Section 9).

Schedule optimisation. Align Your Steps (AYS) [Sabour et al., 2024] optimises the sampling timesteps for a chosen model, dataset, and stochastic solver by minimising a bound on the mismatch between a locally linearised and true reverse process. It asks *where* expensive evaluations should occur. Latent teleportation asks *what state should fill the interval between them*. These decisions are complementary, and Section 5.4 measures their joint offline proxy rather than assuming uniform skips.

Flow matching, straightening, and reflow. Flow matching can train vector fields along diffusion or optimal-transport probability paths; the latter are straighter and admit efficient ODE sampling [Lipman et al., 2023]. Rectified flow and reflow make this connection operational by straightening noise-to-data trajectories, which in turn eases few-step distillation [Liu et al.,

2023, 2024b]. This is the geometry latent teleportation tries to exploit after training: straighter paths make a local tangent valid for longer, while schedule calibration absorbs the remaining non-uniform speed. Reflow changes the model; teleportation can be fitted to a frozen model. Combining them should improve predictability, but must be re-measured because a successfully straightened few-step model also leaves fewer calls to remove.

Training-free caching. DeepCache [Ma et al., 2024] reuses U-Net skip features across adjacent steps. FORA [Selvaraju et al., 2024] caches attention and MLP outputs in diffusion transformers. TeaCache [Liu et al., 2024a] uses changes in timestep-modulated inputs as a cheap signal for dynamic reuse. TaylorSeer [Liu et al., 2025] reframes caching as *forecasting*: rather than reusing a stale feature, extrapolate it with a Taylor series in the timestep index. Confidence-Gated Taylor [Guan et al., 2025] moves the forecast to the last block and uses the first block’s observed error to decide whether to reuse or fully compute. Crucially, these methods are fitted or calibrated to a specific model: the cache schedule and the expansion order are tuned per checkpoint, and transfer is not claimed. Our approach inherits that property and we make it explicit, since it is a real deployment cost rather than an incidental detail.

Speculative decoding. Draft-and-verify [Leviathan et al., 2023, Chen et al., 2023], with later variants that avoid a separate draft model by adding heads [Cai et al., 2024], reusing features [Li et al., 2024], or drafting from the prompt itself [Saxena, 2023]. The property that makes these work is verification: the target model’s distribution is preserved exactly. It is worth stating plainly that *our diffusion analogue does not have this property*. There is no cheap exact verifier for “is this latent the one the big model would have produced”, so every method in this paper is an approximation with a quality cost, and must be evaluated as such.

3 Method

3.1 Speculative latent walking

The drafting scheme mirrors speculative decoding structurally. The big model takes one real step; a small network drafts the next k ; a second network corrects the endpoint of the draft onto where the big model’s trajectory would plausibly have gone; the loop resumes with a real step.

The **latent walker** W predicts the next latent from the current one and the trajectory’s momentum $\delta_i = x_i - x_{i-1}$:

$$\hat{x}_{i+1} = x_i + \delta_i + W(x_i, \delta_i, t_i), \quad (2)$$

so that the network learns only the *correction* to constant-velocity motion. The output convolution is zero-initialised, making the network exactly a momentum extrapolator at initialisation and guaranteeing it starts no worse than the training-free baseline. Rolling W out k times, feeding it its own predicted movement, yields a draft endpoint $\hat{x}_{i+k}$.

The **gap interpolator** G then teleports that endpoint:

$$\tilde{x}_{i+k} = \hat{x}_{i+k} + G(x_i, \delta_i, \hat{x}_{i+k}, t_i, k), \quad (3)$$

again residual and zero-initialised, so identity means “trust the walker”. Both networks are small FiLM-conditioned residual CNNs operating directly on the 16-channel latent, conditioned on sinusoidal embeddings of the fractional timestep and the schedule length.

3.2 First-order Taylor forecasting and confidence gating

The training-free baseline extrapolates the trajectory’s own momentum:

$$\tilde{x}_{i+k} = x_i + k(x_i - x_{i-1}), \quad (4)$$

which is exactly a first-order Taylor step in the step index, in the spirit of Liu et al. [2025]. This is the floor every learned method must beat, and in our results it is a stubbornly high floor.

The **confidence gate** follows the motivation of CG-Taylor [Guan et al., 2025]. Rather than a fixed skip schedule, it would observe a cheap partial computation and commit to the forecast only when that signal predicts low error. The end-to-end Taylor arm reported below uses fixed k and does not activate this gate; we therefore treat confidence gating in this codebase as unvalidated system design rather than borrowing the published CG-Taylor speed claims.

3.3 Aligned anchors and selective batch lanes

A constant k assumes that equal scheduler-index gaps are equally easy to forecast. They are not. We separate the decision into an anchor schedule $A = (a_0, \dots, a_m)$ and a transport rule T :

$$\tilde{x}_{a_{j+1}} = T(x_{a_j}, x_{a_{j-1}}, c, a_j, a_{j+1} - a_j). \quad (5)$$

AYS motivates choosing the anchors non-uniformly; our calibrated transport uses an empirical per-step, per-horizon speed correction. A retrieved neighbour can further supply a local tangent or curvature estimate, so similar historical trajectories adjust both the direction of the warp and the next point at which the big model should refine it. This is better understood as moving within a local chart of the latent manifold than as copying a cached image.

For a batch of B outputs, the schedule can be lane-specific: $A^{(1)}, \dots, A^{(B)}$. At a global scheduler event, lanes requiring a real update are compacted into an active microbatch; the remaining lanes advance with T . Different noise seeds already provide ordinary batch diversity. Lane-specific neighbours, horizons, and bounded residual perturbations add *structured* variation without requiring every lane to take every denoiser call. This saves work only if the active-lane compaction costs less than the removed transformer rows; issuing a full batch and masking its outputs saves nothing. The released code includes the deterministic event planner that groups lane endpoints into compact denoiser microbatches; the GPU gather/transformer/scatter path remains to be benchmarked. Section 7 gives the production protocol, while Section 5.4 measures only the offline schedule proxy.

3.4 Latent teleportation from a trajectory cache

The cache stores, per generation, the full sequence of intermediate latents together with a CLIP [Radford et al., 2021] embedding of the prompt and a tokenisation of it into *visual units* — noun-phrase-like fragments that recur across prompts — plus bigrams of adjacent units. Directly copying a neighbour’s latent move entangles its composition with the query. Our retained architecture instead preserves the query momentum and retrieves a residual correction. For neighbour j at step t and horizon h , define

$$r_{t,h}^{(j)} = (x_{t+h}^{(j)} - x_t^{(j)}) - \alpha_{t,h} h (x_t^{(j)} - x_{t-1}^{(j)}). \quad (6)$$

The online forecast is

$$\hat{x}_{t+h} = x_t + w_\ell \alpha_{t,h} h \Delta x_t + w_r \sum_{j \in \mathcal{N}_k} q_j r_{t,h}^{(j)}, \quad (7)$$

where the two gate weights are fitted on training trajectories and q_j are normalized neighbourhood weights.

Retrieval first selects a prompt-similar pool. It then prunes that pool by a weighted combination of prompt similarity and agreement between compact motion descriptors: each observed $16 \times 64 \times 64$ latent delta is average-pooled to $16 \times 8 \times 8$ and cosine-normalized. This two-stage search lets the forecaster use the k trajectories whose recent motion best matches the query, while Equation 7 makes the gate fall back toward local momentum when their corrections do not transfer. We compare this design with prompt-only top- k , direct motion transport,

per-neighbour scalar speed, channel-wise transport, and dense attention in Section 5.5. Storage is SafeTensors blobs indexed by SQLite; FAISS [Johnson et al., 2019] or a serving-native vector index replaces the exact scan at scale.

3.5 Extension to LoRA armies

Production serving does not run one checkpoint. It runs a base model with a library of LoRA adapters [Hu et al., 2022] swapped per request — in our deployment, an army of style and subject adapters, hot-swapped between generations. This breaks the trajectory cache in a specific way: an adapter shifts the latent statistics, so a trajectory collected under adapter A is not a valid warm start under adapter B , even for an identical prompt. Naively sharing the cache produces exactly the failure mode the confidence gate is supposed to catch, but at a magnitude that no gate rescues.

The extension we propose — and have not yet validated, so we state it as design rather than result — is an *embedding normalisation network*: a small conditional map $\eta_{A \rightarrow B}$ trained to transport a latent from adapter A ’s trajectory space into adapter B ’s, conditioned on both adapter identities and the timestep. Cheap variants worth trying first, in increasing order of capacity, are (i) per-adapter per-timestep channel-wise mean/variance renormalisation, which costs two vectors per adapter per step; (ii) a low-rank affine map fitted per adapter pair; (iii) a shared conditional network taking learned adapter embeddings, which scales to $O(n)$ parameters rather than $O(n^2)$ for n adapters. Adapter identity is exactly the kind of low-cardinality conditioning that FiLM handles well, and the zero-initialised residual trick used above applies again: initialise to identity so that a shared cache degrades to a per-adapter cache rather than to garbage.

The adapter library and its serving stack are not open-sourced; the mechanism described here is, in Penkman [2026a].

4 Experimental Setup

All measurements are on a single RTX 5090 (32 GB, sm_120, compute capability 12.0) using Z-Image Turbo in `bf16`, at 512×512 with a 16-step schedule and guidance 0, seed fixed at 7 across arms. Trajectory capture collected 200 full 16-step trajectories (402 MB) storing every intermediate latent. The image ablation uses six deliberately heterogeneous prompt slices: golden-hour landscape, dramatic portrait, snowy night, cyberpunk/neon, soft-light wildlife, and isometric illustration. These labels describe prompt content; they are not independent style datasets or LoRA adapters.

Latency aggregates use five steady-state prompts per configuration. A baseline row is classified as initialization when it exceeds twice the within-run median baseline latency; one such row is present for each draft length and is excluded from timing only. All six saved prompt outputs remain in the image analysis. Relative-error aggregates likewise exclude targets whose recorded movement is below 10^{-4} , because the metric divides by that movement. These rules are fixed for every method before comparison.

Metrics. Forecast quality is measured by

$$\text{relL2} = \frac{\|\tilde{x}_{i+k} - x_{i+k}\|_2}{\|x_{i+k} - x_i\|_2}, \quad (8)$$

error normalised by how far the trajectory actually moved. This normalisation is what makes the number interpretable: $\text{relL2} = 1.0$ is the score of predicting no movement at all, so any method scoring near 1.0 is worthless regardless of its absolute error. End-to-end image agreement is PSNR against the full-schedule baseline image. For the image grids we additionally report SSIM,

computed on the saved RGB PNGs. Both are reference-similarity metrics rather than measures of prompt adherence or aesthetic quality. Wall clock is measured with CUDA synchronisation around each arm.

Reproducibility. Timing and latent-forecast numbers are emitted by `scripts/paper_analysis.py` from raw result JSON. `scripts/spec_forecaster_ablation.py` fits and cross-validates horizons through $k = 8$ and emits full-schedule deployment coefficients; `scripts/spec_schedule_ablation.py` searches anchor compositions under fixed budgets; `scripts/spec_knn_ablation.py` evaluates retrieval architectures and resident-cache overhead; and `scripts/spec_knn_visual.py` decodes the held-out long-warp grid. `scripts/spec_train_cache_adapter.py` trains and evaluates the focused head, while `scripts/package_cache_adapter.py` assembles its weights, schedule table, model card, and metrics. The production helper `forecast_with_pruned_knn_residual` implements the retained cache path. Image tables and grids are emitted by `scripts/paper_ablation_grids.py` from the 72 saved PNGs. The latter writes a manifest containing the SHA-256 digest, prompt index, method, k , PSNR, and SSIM for every source panel. No panel is synthesized or retouched; grid construction is limited to resizing, borders, and labels.

5 Results

5.1 The latent trajectory is forecastable

Table 1 summarises the (t, k) grid of forecast error. Two facts stand out. First, first-order Taylor extrapolation is strong: at $k = 1$ it reaches $\text{relL2} = 0.205$ around step 6, meaning the residual error is about a fifth of the movement being predicted. Second, the scalar-affine predictor sits near 0.88 — barely better than not moving — which tells us the trajectory’s predictability is genuinely directional rather than a matter of getting the step size right.

k	mean relL2 (taylor1)	best cell	mean relL2 (affine)	cells
1	0.240	0.205	0.880	13
2	0.318	0.255	0.879	13
3	0.330	0.261	0.883	12
4	0.369	0.261	0.880	11

Table 1: Trajectory predictability by draft length. relL2 is error over actual movement; 1.0 is the no-movement baseline. Lower is better. The degenerate final-step cell is excluded (see text).

Error degrades monotonically with k , as expected, and is worst at both ends of the schedule: early steps move erratically, and the last scheduler step of this 16-step configuration moves the latent by approximately zero. That final step is not merely hard to predict — it is a no-op, and dividing by its movement produced an relL2 of 5.48×10^9 at $t = 14$. We exclude that cell from all aggregates. In the stored float16 trajectories, the final latent is duplicated; for this exact pipeline and schedule, that step is a candidate for exact removal after verifying the duplication at runtime.

5.2 Forecaster screening

Training the walker and interpolator for 12 epochs on the 200 captured trajectories reaches a best relL2 of 0.242 for the walker, against roughly 0.21–0.30 for Taylor over the comparable window — an improvement of order 15–20% on its training metric. Momentum conditioning provides the useful signal. Validation selection is required: the best walker checkpoint occurs

before the final epoch. The cheaper calibrated forecaster in the next section ultimately provides the stronger deployment point because it captures the schedule effect without a second neural network allocation.

5.3 Schedule calibration improves the cheap forecaster

The basic Taylor rule assumes that one recorded movement should be multiplied by exactly k . Diffusion schedules are not uniform in trajectory distance, so we fit a separate scalar $\alpha_{t,k}$ for each step and horizon:

$$\hat{x}_{t+k} = x_t + \alpha_{t,k} k (x_t - x_{t-1}). \quad (9)$$

We also test a two-delta fit $x_t + a_{t,k}(x_t - x_{t-1}) + b_{t,k}(x_{t-1} - x_{t-2})$, an average-velocity rule, and a direct second-order Taylor extrapolation. Coefficients are fitted on four folds and evaluated on the held-out fifth, rotating over all five folds of the 200 captured trajectories. Cells whose target movement is below 10^{-4} are excluded by the same rule as Section 5.1.

k	Taylor-1	avg. velocity	Taylor-2	scaled momentum	two-delta fit
1	0.221±0.005	0.273±0.004	0.226±0.009	0.176±0.007	0.172±0.007
2	0.306±0.004	0.338±0.004	0.370±0.011	0.205±0.007	0.201±0.007
3	0.317±0.004	0.352±0.004	0.395±0.013	0.228±0.008	0.226±0.008
4	0.357±0.004	0.393±0.004	0.446±0.016	0.250±0.009	0.248±0.008
5	0.402±0.004	0.435±0.004	0.505±0.018	0.271±0.009	0.269±0.009
6	0.445±0.004	0.473±0.003	0.567±0.020	0.291±0.010	0.289±0.010
7	0.484±0.004	0.510±0.003	0.628±0.021	0.310±0.010	0.309±0.010
8	0.521±0.003	0.544±0.003	0.689±0.022	0.330±0.011	0.327±0.011

Table 2: Five-fold held-out relL2 (mean $\pm$ standard deviation across folds). Lower is better. Scaled momentum fits one schedule scalar; two-delta fits two. No image pixels or timing labels enter the fit.

Table 2 gives a constructive tuning result. Scaled momentum reduces error by 20–37% relative to first-order Taylor across $k = 1 \dots 8$, with the largest relative gain at $k = 8$ (36.7%). The two-delta fit remains only marginally better: at $k = 8$ it reaches 0.327 against 0.330 for one scalar. The second delta therefore contributes little even at long horizons, making the one-scalar rule the better first deployment candidate. Average velocity and raw second-order Taylor are both worse than first order: more history or a higher expansion order is not automatically better unless it is calibrated to the schedule. This is a held-out latent-level result, not yet an image-quality or latency claim; the next image sweep should promote scaled momentum alongside the existing Taylor reference.

5.4 Aligning anchors improves a fixed interval budget

We next hold the number of forecast intervals fixed and search their integer horizons. The proxy covers recorded steps 2 through 14, excluding the duplicated final latent. For each outer fold, coefficients and a schedule are selected on four folds and evaluated on the fifth. Each interval starts from its recorded true anchor; the experiment therefore measures local anchor placement and does not simulate error accumulated by a branched sampler.

Table 3 separates two gains. With four intervals, calibration alone improves uniform scheduling from 0.332 to 0.243. Aligning the same interval budget further improves it to 0.215—35.3% below uniform Taylor and about 12% below uniform calibrated momentum. Every fold selects $[8, 1, 1, 2]$: one long early warp followed by dense late corrections. That pattern is consistent with a rectified-flow path that is relatively straight early but becomes more sensitive as image details settle; it is an empirical result for this schedule, not a universal timetable.

The four best distinct schedules average 0.220, only 2.4% above the single best schedule. This is the first quantitative support for staggered batch lanes: four outputs can use different anchor

intervals	uniform Taylor	uniform calibrated	aligned calibrated	four staggered	selected horizons
3	0.380±0.004	0.270±0.009	0.248±0.008	0.254±0.008	8-1-3
4	0.332±0.005	0.243±0.008	0.215±0.007	0.220±0.007	8-1-1-2
6	0.280±0.005	0.212±0.008	0.188±0.006	0.190±0.006	7-1-1-1-1-1

Table 3: Five-fold anchor-schedule ablation. The budget counts forecast intervals, not denoiser calls. Entries are mean local relL2 $\pm$ standard deviation. Four staggered reports the mean of the four best distinct training-fold schedules, modelling four batch lanes. Selected horizons are stable across all five folds. Lower is better.

patterns with little latent-proxy penalty. It does not yet establish either decoded-image diversity or GPU speedup; both require active-lane compaction and the image protocol in Section 7.

5.5 Pruned trajectory residuals improve the local forecast

We evaluate the retrieval architecture on the same 200 trajectories and five outer folds. Test prompts retrieve only training-fold trajectories. A prompt-similar pool of 16 is pruned to $k \in \{1, 2, 4, 8\}$ using the prompt and motion score from Section 3.4. The grid covers horizons $h \in \{1, 2, 4, 8\}$ and every valid scheduler step. Two gate coefficients are fitted independently for each training fold, step, horizon, and k .

architecture	neighbours	relL2	change vs. local
Calibrated local	–	0.224±0.008	+0.0%
Prompt top- k move	4	0.670±0.039	-66.6%
Motion-pruned move	4	0.565±0.011	-60.5%
Gated residual	1	0.220±0.008	+1.6%
Gated residual	2	0.219±0.008	+2.2%
Gated residual	4	0.217±0.008	+2.8%
Gated residual	8	0.217±0.008	+3.3%

Table 4: Five-fold held-out retrieval architecture ablation across scheduler steps and horizons 1, 2, 4, 8. Positive change means lower error than calibrated local momentum. Direct-move rows test neighbour copying; gated residual rows use Equation 7. Lower is better.

Table 4 identifies a useful architecture boundary. Historical trajectories should condition the correction, not replace the query’s motion. The gated residual improves monotonically with neighbourhood size and reaches relL2 = 0.217 at $k = 8$, a 3.3% gain over schedule-calibrated local momentum. The gate retains the query’s spatial direction and uses neighbours to predict systematic bending away from it. Prompt-only and motion-pruned absolute transport lose that invariant; pruning improves their estimate, but residual fusion is the mechanism that makes retrieval productive.

resident CPU predictor	median ms/query
Local calibrated	0.043
Prompt top-8	0.821
Motion-pruned top-8	0.802
Dense trajectory attention	5.838

Table 5: Resident-cache single-query predictor overhead on CPU with one PyTorch thread (median of 30 repeats). The bank contains 199 trajectories; model inference, storage I/O, and VAE decode are excluded. Lower is better.

The online predictor is small compared with one denoiser evaluation. At $k = 8$, motion-

pruned retrieval takes 0.80 ms/query in the resident CPU microbenchmark, while dense attention over the trajectory bank takes 5.84 ms. Candidate pruning therefore reduces retrieval transport overhead by $7.3\times$ and bounds the number of latent records that must be prefetched. In a GPU server, the same plan maps to a small embedding/motion top- k followed by one weighted residual reduction.

5.5.1 Variance shrinkage and a focused learned adapter

Neighbour disagreement is itself a useful input. A training-only grid chooses a Wiener-style elementwise reliability shrinkage,

$$\bar{r}_{\text{shrink}} = \bar{r} \frac{\bar{r}^2}{\bar{r}^2 + \lambda \sigma_r^2 + \epsilon}, \quad (10)$$

where $\bar{r}$ and σ_r are the weighted mean and standard deviation of the eight retrieved residuals. The selected $\lambda = 0.1$ suppresses cache corrections where neighbours disagree while preserving stable spatial detail.

We also train a 7,312-parameter convolutional adapter. Its inputs are the scalar-gated base move, $\bar{r}$, σ_r , and six schedule/ confidence scalars. A 1×1 projection, two depthwise spatial-mixing blocks, and a zero-initialized output projection predict only the correction to the scalar gate. Zero initialization therefore exactly recovers the best non-neural cache rule.

horizon	calibrated local	scalar cache gate	learned adapter
1	0.1835	0.1801	0.1797
2	0.2110	0.2062	0.2043
4	0.2601	0.2520	0.2481
8	0.3439	0.3257	0.3195

Table 6: Fold-0 held-out relL2 for the focused cache adapter, trained on 160 trajectories and evaluated on 40 disjoint trajectories. The adapter is one shared head across all listed horizons. Lower is better.

The adapter improves every horizon and reaches aggregate relL2 = 0.223, 1.1% below the fitted scalar cache gate and 4.1% below calibrated local momentum. Training takes 132 seconds on CPU; the head itself takes 3.28 ms per $16 \times 64 \times 64$ latent in the one-thread CPU benchmark. The RTX 5090 attempt was safely deferred because co-resident production services left less than 2.5 GiB allocatable VRAM, so we do not substitute an estimated GPU training time. The 31 KB weights, config, schedule coefficients, and raw metrics are published at <https://huggingface.co/lee101/zimage-latent-teleport-cache-adapter>.

endpoint forecast	PSNR (dB)	SSIM
Calibrated local	21.48	0.7731
Pruned 8-NN residual	21.84	0.7881
Variance-shrunk cache	21.84	0.7883
Learned cache adapter	21.92	0.7877

Table 7: Decoded held-out endpoint forecasts for the step-6 $\rightarrow$ 14 long warp, averaged over six prompts from fold 0. Shrinkage has the highest SSIM; the adapter has the highest PSNR. Metrics compare each decoded forecast with the decoded recorded endpoint. Higher is better.

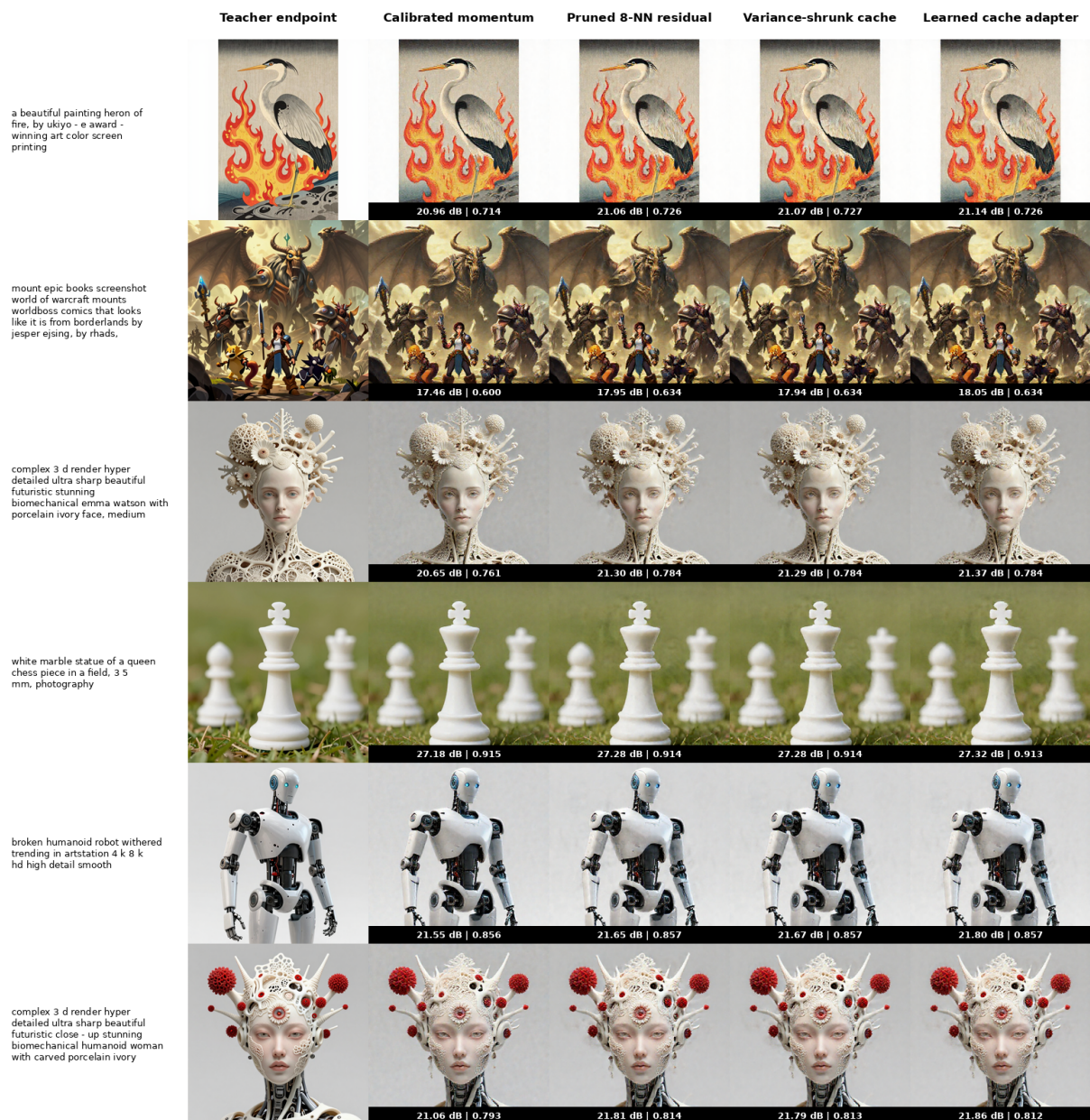


Figure 1: Held-out long-warp procedure ablation. Every row is a test-fold prompt; columns compare calibrated momentum, the eight-neighbour scalar gate, variance-aware shrinkage, and the learned cache adapter. Panels are direct VAE decodes of recorded or forecasted endpoint latents with no content editing.

5.6 End-to-end latency is fixed-cost limited

Table 8 measures the current serving point. Across draft lengths $k \in \{1, 2, 3\}$, corresponding to 9, 6 and 5 real denoiser calls out of 16, measured speedup never exceeds $1.10\times$ and speculative walking specifically never exceeds $1.04\times$.

k	big steps	call red.	wall-clock speedup			PSNR vs baseline (dB)		
			spec	taylor	skip	spec	taylor	skip
1	9	$1.78\times$	1.02	1.08	1.03	18.9	20.8	11.0
2	6	$2.67\times$	1.03	1.03	1.04	18.0	17.9	9.3
3	5	$3.2\times$	1.04	1.09	1.10	17.0	16.9	8.9

Table 8: Draft-length ablation at 512^2 , 16 steps, 5 prompts per cell after warm-up exclusion. “call red.” is the reduction in denoiser evaluations; speedup is measured wall clock. *skip* performs no correction at all and therefore bounds what any drafting scheme can achieve in this harness.

The diagnostic is the *skip* column. That arm discards $3.2\times$ of the denoiser evaluations and replaces them with nothing — its PSNR of 8.9 dB confirms the output is destroyed — and it still only reaches $1.10\times$. Its observed ceiling is therefore about $1.10\times$ at this operating point, implying that roughly 90% of the measured time is outside the removed calls. Inspecting the harness confirms it: the timed region wraps `zimage_prepare` (text encoding) and `zimage_decode` (VAE decode) along with the sampling loop, and at 512^2 with a 16-step Turbo schedule those fixed costs dominate. This is Amdahl’s law [Amdahl, 1967] arriving exactly on schedule.

A second observation from Table 8: at $k = 1$ the training-free Taylor arm is both faster ($1.08\times$ vs $1.02\times$) and higher quality (20.8 dB vs 18.9 dB) than the learned walker. The learned networks win on the latent metric they were trained for but lose on same-seed pixel retention. This sets a useful tuning reference: Taylor at $k = 1$ is the current image-level baseline, and a learned replacement needs an image-aligned objective and validation-based checkpoint selection to displace it.

5.7 Native exact replay, quantisation, caching, and fusion

Approximate forecasting is only one tier of the serving problem. When prompt, negative prompt, size, step count, guidance, seed, and resume step all match a previous request, the native backend can resume from the stored intermediate latent without predicting anything. The cache key includes all of those fields, the value is a CPU-resident opaque `sd_latent_t`, and a bounded LRU controls host memory. Table 9 compares an unsplit reference, a split prime that populates the cache, and the following exact hit. All three pixel hashes match within every row.

case	size	baseline (s)	prime (s)	replay (s)	prime/replay
product_glass_robot	512x512	21.40	23.85	19.84	$1.202\times$
pixel_game_asset	512x512	23.41	23.78	18.31	$1.299\times$
portrait_hands	512x768	25.73	25.75	21.16	$1.217\times$

Table 9: Exact latent replay on the live shared RTX 5090 service with a 9-step Z-Image-Turbo Q4_K backend. Speedup compares the split prime with its cache-hit replay; the unsplit baseline is the exactness reference. Shared-load timings are paired observations, not idle-device throughput.

The paired median is $1.217\times$. A separate four-hit soak passed every cache-hit and pixel-hash gate without OOM. Its speed ratio was $0.978\times$ under heavier contention, so it is retained as a stability result and not pooled into the speed estimate. A fresh-key memory probe sampled

the native server at 3824 MiB after the prime and 1818 MiB after each of two subsequent hits. These coarse post-request samples show a plateau rather than an allocator-level peak, but they directly rule out monotonically retained GPU state across those repetitions.

A subsequent resume-step sweep tests how much of the deterministic trajectory to retain. Table 10 uses one 512^2 case and two replays per prime. All five settings are pixel exact. The latest checkpoint, step 8 of 9, is fastest in the paired sweep at $1.530\times$ and is then promoted with fresh seed keys across three cases. Those primes are explicitly required to be cache misses; the three replay speedups are $1.331\times$, $1.491\times$, and $1.583\times$, for a $1.491\times$ median.

resume step	prime (s)	replay 1 (s)	replay 2 (s)	prime/median replay
2	29.12	27.76	23.36	$1.139\times$
4	24.96	24.31	24.09	$1.031\times$
6	23.46	18.19	17.75	$1.305\times$
7	29.45	21.11	19.36	$1.455\times$
8	29.40	17.54	20.89	$1.530\times$

Table 10: Exact replay resume-step sweep on the live shared RTX 5090. Every prime is the first request for its step-specific key and each replay hash matches its unsplit reference. Shared-load variance makes the intermediate ordering noisy, while the latest exact state consistently saves the most remaining denoiser work.

The serving default is therefore dynamic **steps-1**, rather than a fixed step 7. The benchmark adds a seed-offset cache buster, rejects an unexpectedly warm prime, and retries only transient 429/503 admission responses. These gates were necessary: without them, a hit-versus-hit rerun briefly appeared to erase the speedup, and shared-GPU backpressure aborted another otherwise valid arm.

Compression also has a non-monotonic speed trade. The historical RTX 3090 sweep in Table 11 found Q8_0 faster than Q4_K despite its larger memory footprint; Q6_K was slower and failed on its fifth generation. The current shared 5090 profile therefore uses Q4_K for residency headroom, not because lower-bit weights measured faster.

GGUF quant	seconds/iteration	VRAM (GiB)	outcome
Q8_0	1.02	10.7	completed
Q4_K	1.07	8.1	completed
Q6_K	1.20	9.4	crashed on generation 5

Table 11: GGUF quantisation ablation on RTX 3090 at batch 1 and 1024^2 . Seconds/iteration exclude HTTP and image encoding. Q4_K saves 2.6 GiB but is about 5% slower than Q8_0; Q6_K is neither a speed nor a reliability win.

The native step-cache sweep in Table 12 gives a second negative result. DBCache makes six steps faster, but its 15.12 dB retention is effectively the three-step 15.07 dB point at 40% more latency. TaylorSeer is a no-op in this particular stable-diffusion.cpp Z-Image sampler path and pays bookkeeping overhead. This is an implementation- and operating-point result, not a contradiction of feature forecasting on the models evaluated by Liu et al. [2025].

method	steps	median (s)	PSNR vs. 20-step (dB)
none	3	5.81	15.07
none	6	11.64	17.27
TaylorSeer	6	11.87	17.27
DBCACHE	6	8.16	15.12
TaylorSeer	10	18.73	20.25

Table 12: Native step-cache ablation at 1024^2 on RTX 3090. Absolute blocks include thermal drift, so comparisons are confined to the sweep. On this already-distilled Turbo schedule, three uncached steps dominate six-step DBCACHE at nearly the same same-seed retention.

Finally, the new sm_120 residual-add plus RMS-normalisation epilogue reduces the isolated production-shape kernel from 0.298 to 0.171 ms, a $1.74\times$ microbenchmark speedup, and passes both kernel and transformer-block reference tests. It remains opt-in until full-pipeline image parity and end-to-end timing pass: 30 uses of a sub-millisecond epilogue do not justify multiplying the isolated ratio into a generation claim.

The OOM audit also exposed an architectural rather than allocator failure in the experimental neural latent combiner. Flattening a $16 \times 64 \times 64$ latent into four dense layers implied 4.50 billion parameters, so constructing a second instance could consume the host before inference began. The replacement adaptively pools the latent pair to rank 64, conditions that state on the text embeddings, and decodes only a zero-initialised residual around the arithmetic mean. It has 4,600,064 parameters, a $979\times$ reduction, and its 12-test production-shape suite peaks at 496 MiB RSS. This is the useful form of a pruned auxiliary subnet here: factor the state before training rather than quantising an unnecessarily dense graph.

5.8 Image ablations across methods, styles, and draft lengths

The aggregate timing result is bottleneck-limited, and the images answer the complementary question of *how* each approximation changes the sample. Figure 2 gives the compact method comparison, while Appendix A exposes the complete method $\times$ draft-length $\times$ prompt matrix rather than one favourable prompt. The raw store contains 72 PNGs: six prompts, four arms, and three values of k . Because the baseline is regenerated identically for every k , this corresponds to six unique references and 54 candidate outputs. The figures include all six prompt slices, including the first timing row excluded as a warm-up outlier; warm-up affects latency, not the already-saved image.

k	real calls	learned		Taylor		skip	
		PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
1	9	18.14	0.699	19.78	0.770	11.11	0.244
2	6	17.28	0.662	17.29	0.637	9.35	0.182
3	5	16.51	0.629	16.39	0.585	8.95	0.173

Table 13: Image hyperparameter ablation over all six prompts. Values are mean PSNR (dB) and SSIM against the same-seed 16-step reference. Unlike Table 8, no row is removed because this table contains no timing measurement. Higher is better.

Table 13 gives three robust observations. First, $k = 1$ is the quality frontier: Taylor reaches 19.78 dB/0.770 and the learned method 18.14 dB/0.699. Second, mean similarity falls as k grows. Learned SSIM moves $0.699 \rightarrow 0.662 \rightarrow 0.629$, while Taylor moves $0.770 \rightarrow 0.637 \rightarrow 0.585$. Third, the skip control is not a plausible alternative generation: even at $k = 1$ it reaches only 0.244 SSIM and visually collapses to coloured noise. It is useful only as the compute-saving upper bound in Table 8.

The style-stratified $k = 3$ numbers in Table 14 show why six rows matter. Learned speculation is ahead of Taylor on portrait, snowy-night, and isometric prompts; Taylor is ahead on landscape, cyberpunk, and wildlife by PSNR. The ranking is therefore three against three rather than a universal winner. Structural retention also varies widely: learned SSIM ranges from 0.500 on the snowy scene to 0.791 on the isometric illustration. These are descriptive slices, not confidence intervals, but they rule out a claim that one forecaster dominates across visual regimes.

style slice	learned		Taylor		skip	
	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
Golden-hour landscape	13.87	0.713	14.12	0.666	9.32	0.165
Dramatic portrait	19.11	0.513	17.72	0.453	7.37	0.096
Snowy night scene	15.26	0.500	14.68	0.454	8.82	0.177
Cyberpunk / neon	14.70	0.504	15.74	0.506	9.33	0.217
Soft-light wildlife	18.53	0.751	18.82	0.692	11.79	0.284
Isometric illustration	17.61	0.791	17.28	0.738	7.08	0.097

Table 14: Style-slice ablation at $k = 3$ over the six recorded prompts. Each row contains one image per method, so the values describe examples rather than a population estimate.

An oracle that independently chooses the higher-scoring learned or Taylor image for each prompt estimates the available headroom from method routing. It is not a deployable router because it sees the reference image and chooses PSNR and SSIM independently.

k	PSNR (dB)			SSIM		
	learned	Taylor	oracle	learned	Taylor	oracle
1	18.14	19.78	19.78	0.699	0.770	0.772
2	17.28	17.29	17.62	0.662	0.637	0.665
3	16.51	16.39	16.78	0.629	0.585	0.629

Table 15: Reference-aware oracle routing between the learned and Taylor candidates. The oracle is an unavailable upper bound, not an evaluated routing algorithm. Higher is better.

Table 15 shows modest routing headroom. At $k = 1$, Taylor already wins PSNR on every prompt; at $k = 2$ and $k = 3$, oracle selection adds only 0.33 and 0.38 dB over the better fixed method. Oracle SSIM is similarly close to the best fixed choice. Schedule calibration is therefore the more promising immediate experiment; a learned router becomes worthwhile only when the candidate methods are more complementary.

The grids also sharpen the metric caveat. A coherent candidate that moves a subject or changes texture can score poorly against the reference, while SSIM rewards structurally simple scenes differently from detailed photographs. We therefore use these metrics to quantify same-seed retention, not to assert equal aesthetic quality. A blinded human or calibrated vision-language preference study remains required for that claim.

We additionally inspected every full-resolution source panel, not only the contact sheets. In this unblinded author screen, all 36 learned and Taylor candidates retained the requested subject or scene, with no catastrophic collapse; increasing k primarily introduced texture, geometry, and layout drift. None of the 18 skip candidates passed the same usable-image screen, although several $k = 1$ panels retained fragments of scene structure. This is a failure-mode check, not a blinded aesthetic or prompt-adherence evaluation.

The complete grid and larger method-specific sheets are provided in Appendix A; the separation keeps panel labels readable while retaining every recorded output.

Method × prompt/style ablation (seed 7)

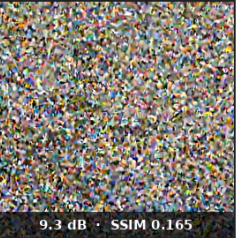
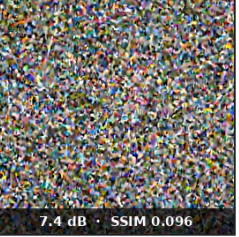
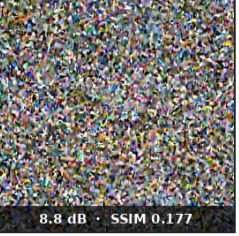
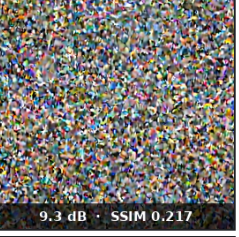
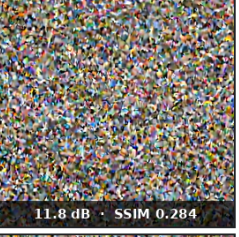
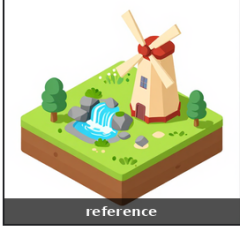
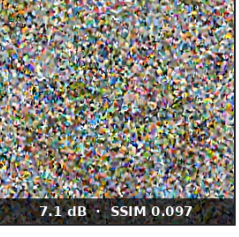
	Baseline 16 real steps	Learned walker + interpolator $k=3$ / 5 calls	Taylor momentum $k=3$ / 5 calls	Skip control $k=3$ / 5 calls
Golden-hour landscape a lighthouse on a cliff at golden hour, crashing waves	 reference	 13.9 dB · SSIM 0.713	 14.1 dB · SSIM 0.666	 9.3 dB · SSIM 0.165
Dramatic portrait portrait of an old fisherman with a pipe, dramatic lighting	 reference	 19.1 dB · SSIM 0.513	 17.7 dB · SSIM 0.453	 7.4 dB · SSIM 0.096
Snowy night scene a cozy cabin in a snowy forest at night, warm windows	 reference	 15.3 dB · SSIM 0.500	 14.7 dB · SSIM 0.454	 8.8 dB · SSIM 0.177
Cyberpunk / neon cyberpunk street market in the rain, neon signs	 reference	 14.7 dB · SSIM 0.504	 15.7 dB · SSIM 0.506	 9.3 dB · SSIM 0.217
Soft-light wildlife a fox curled up on autumn leaves, soft light	 reference	 18.5 dB · SSIM 0.751	 18.8 dB · SSIM 0.692	 11.8 dB · SSIM 0.284
Isometric illustration isometric tiny island with a waterfall and windmill	 reference	 17.6 dB · SSIM 0.791	 17.3 dB · SSIM 0.738	 7.1 dB · SSIM 0.097

Figure 2: Method × prompt/style ablation at $k = 3$ (five real denoiser calls), seed 7. Each row is one saved prompt; each panel is an actual generated PNG with no content editing. The learned and Taylor arms usually preserve the requested subject while changing layout, texture, or fine detail. The skip control is visibly destructive in every slice.

6 Discussion

The ablations support the following deployment decisions.

1. **Serve exact repeats from bounded latent replay first.** It is the only measured tier with no quality trade: all tested pixels match, the fresh-key paired median is $1.491\times$, and the repeated-hit memory probe plateaus. Keep the full generation configuration in the key, keep opaque latents in host memory, and use a bounded LRU.
2. **For unseen prompts, tune from Taylor at $k = 1$.** It is the best measured image speed/retention point: $1.08\times$ at 20.8dB, with no learned checkpoint or extra model residency. The learned walker and larger k values remain experimental.
3. **Promote calibrated momentum and aligned anchors together.** Schedule calibration lowers five-fold trajectory error by 20–37% through $k = 8$, and anchor search adds another latent-proxy gain. Refit both for every model, resolution, solver, and step count; the current image-level evidence remains raw Taylor, not the calibrated rule.
4. **Retrieve small residuals only when the horizon justifies it.** Keep the prompt pool and full records in host memory, motion-prune to eight neighbours, and prefetch only their residuals. The decoded gain is modest; neither the 7,312-parameter adapter nor dense attention earns default GPU residency at this operating point.
5. **Choose the memory profile independently from the speed profile.** Q8_0 measured faster when it fit; Q4_K is the shared-GPU headroom choice. Bound Python pipeline variants to one by default, clear evicted references, retain VAE tiling/offload controls, and admit full-BF16 or compiled variants only under a device-memory lease. Keep the fused residual–RMS path opt-in until full-image parity and end-to-end timing pass. Use the rank-64 latent combiner when a learned merge is required; the dense flattened form is prohibited by construction.
6. **Gate every promotion with a phase-separated, repeated benchmark.** Warm every arm, rotate execution order, cache prompt embeddings only when the workload permits it, and report prompt encode, denoiser, VAE decode, end-to-end latency, peak VRAM, OOM count, PSNR, SSIM, perceptual score, and the complete image grid. Also test the duplicated final latent behind a runtime equality/tolerance guard rather than assuming the result transfers to another schedule.

7 Production Use Cases and Composability

7.1 Thirty-step quality at approximately four-call speed

The production objective is not merely to make a native four-step sample; it is to approach the quality of a 30-step teacher while paying for approximately four large-denoiser evaluations. We treat this as the next acceptance target, not as a result already established by the 16-step study. The experiment should capture full 30-step trajectories for multiple prompts and seeds, fit schedule-specific momentum and neighbour-transport parameters, choose four anchors on training folds, and then run a genuinely branched sampler on held-out prompts. Capture must include the initial pre-denoiser noise latent, which the current callback-only 16-step store omits, so the first warp can be calibrated rather than falling back to raw Taylor.

Three references are required: native four-step generation, the full 30-step teacher, and four-call teleportation from the same initial noise. A candidate passes only if (i) denoiser calls and both denoiser-only and end-to-end latency are reported; (ii) every decoded panel is screened for collapse; (iii) PSNR, SSIM, and a perceptual metric quantify teacher retention; and (iv)

blinded preference and prompt-adherence evaluation cannot distinguish it materially from the 30-step quality mode. A high PSNR alone is not the goal: a useful system may produce a different, equally strong composition.

The training objective should likewise combine endpoint consistency with decoded perceptual supervision:

$$\mathcal{L} = \lambda_z \|\tilde{x}_{a_{j+1}} - x_{a_{j+1}}\|_2^2 + \lambda_p \mathcal{L}_{\text{perc}}(D(\tilde{x}), D(x)) + \lambda_c \mathcal{L}_{\text{consistency}}, \quad (11)$$

where D is the VAE decoder. This connects the walker to consistency distillation without requiring the production denoiser itself to be replaced.

7.2 Four varied images with selective refinement

A common serving request generates four candidates for one prompt. Text encoding, model residency, and launch overhead are already shared, while four independent noise seeds provide baseline diversity. Latent teleportation adds a controlled axis: assign each lane a different high-ranking anchor schedule and, when available, a different compatible cached-neighbour prior. The four near-optimal schedules in Table 3 cost only 2.4% more latent error than the best single schedule in the offline proxy, so staggering need not mean choosing obviously weak paths.

At each anchor, the server compacts only lanes needing the big denoiser into a microbatch. Teleported lanes continue through scalar momentum, a resident small walker, or prefetched neighbour deltas. A later shared anchor can reconverge the lanes for an efficient full batch. This creates a natural quality–variation–compute control: conservative lanes receive more real refinement, exploratory lanes receive a different warp, and all four reuse the same prompt state. The production benchmark must report batch latency, active-lane sizes, pairwise perceptual diversity, prompt adherence, and per-lane quality; the current paper has not yet measured that GPU path.

7.3 How the accelerators combine

The methods operate at different layers, so several combinations are natural. They are not automatically multiplicative: once one method removes most denoiser work, Amdahl’s law reduces the headroom of the next.

8 Serving Integration

The intended deployment target is a single-GPU multi-tenant server in which a diffusion worker shares a device with a language model, a background-removal worker and a 3D worker. Two pieces of that stack are relevant here and are open source [Penkman, 2026b].

First, admission and device-memory arbitration. Four processes sizing their own workloads from `cudaMemGetInfo` is a race: the read counts a caching allocator’s retained blocks as used, and is stale the instant two tenants read it concurrently, so both size a batch against the same free bytes and the second runs out of memory. The gateway replaces this with a lease broker — a tenant asks for headroom and is told a number it may rely on, with granted-but-not-yet allocated memory withheld from the next caller. This is what would allow a draft model to be resident at all, which is the natural next step for the speculative family.

The accompanying Python loader now bounds base and ControlNet pipeline caches to one configuration each by default. Changing dtype, compile mode, offload mode, device, or model no longer silently retains every multi-gigabyte variant; LRU eviction drops references, runs collection, and returns unused CUDA blocks. Setting the bound to zero disables reuse, and an explicit clear function supports test isolation and low-memory recovery. The native exact-replay cache is a different object: it stores bounded CPU latents, so a cache hit does not pin a second transformer or grow with resolution on the GPU.

Technique	What it contributes	Combination with latent teleportation
AYS	Places a small number of solver evaluations where their local approximation matters most.	Supplies the real anchors; teleportation predicts the latent state between them.
Flow matching / reflow	Trains straighter probability-flow trajectories and improves low-NFE integration.	Extends the horizon over which a tangent or neighbour transport remains valid; coefficients must still be refit.
Consistency / LCM	Learns a direct few-step map to a consistent endpoint.	Acts as a cheap proposer, endpoint teacher, or fallback lane; a native 2–4-step model leaves less post-training skip headroom.
DeepCache / FORA / TaylorSeer	Reuses or forecasts internal denoiser features across nearby steps.	Can reduce the cost of the remaining real anchors while latent teleportation reduces their count.
Fused kernels / compiled serving	Reduces each real call, allocation, transfer, and launch overhead.	Speeds the unavoidable anchors and makes active-lane compaction practical; measured total gain remains Amdahl-limited.
Trajectory retrieval	Provides prompt- and style-conditioned local directions from previous generations.	Adjusts the warp direction, curvature, and confidence rather than copying a finished image.

Table 16: Composability map. Rows describe mechanisms and hypotheses, not a benchmark claiming that all combinations have been measured.

The calibrated forecaster itself needs only a small table of $(t, k, \alpha_{t,k})$ scalars and should remain device-resident. Larger trajectory records belong in pinned host memory or NVMe, with prompt retrieval and asynchronous prefetch occurring while the text encoder or preceding anchor runs. The request scheduler should batch by model/LoRA identity first, then by next anchor, compact active lanes into contiguous buffers, and release cached trajectory pages as soon as their last consumer advances. Model or adapter swaps become natural preemption boundaries: finish an anchor, spill the small latent state, serve the higher-priority resident model, then restore and continue from a verified point.

Second, the request path itself is C rather than Python, with the HTTP relay preserving byte-exact streaming so that image bytes are never re-encoded. CuteDSL contributes fused kernels used by the Z-Image transformer on sm_120 — fused QKV projection with RoPE, fused QK-norm, RMS norm, SiLU-gated FFN and AdaLN — which accelerate the denoiser itself and are therefore complementary to everything in this paper: they make each step cheaper, whereas we tried to make steps fewer. Given the Amdahl result of Section 5.6, kernel work of this kind is currently the better investment of the two at our operating point. The new residual-RMS epilogue is deliberately excluded from that default set until its full-pipeline gate passes, despite the positive microbenchmark in Section 5.7. At a 30-step quality mode, however, denoising should occupy a much larger share of latency; step reduction, per-step fusion, and memory-aware scheduling can then compound over different portions of the request.

9 Limitations and Threats to Validity

The base model is already distilled. Z-Image Turbo is a few-step model. Step-reduction methods have far less to remove from an already-compressed schedule than from a 50-step sampler, and results here should not be extrapolated to undistilled checkpoints, where we would expect drafting to look considerably better.

Small prompt and seed set. Six prompts per configuration, one seed, and five prompts after warm-up exclusion for timing. The contact sheets expose every image rather than hiding this sample size, but they do not turn six examples into a population. This is enough to establish that the timing effect is near $1.0\times$ and nowhere near enough to rank methods separated by a few percent. The method-by-style reversals at $k = 3$ are evidence against a universal ranking, not evidence for six distinct style distributions.

PSNR and SSIM are retention metrics, not quality judges. A drafted trajectory that lands on a different but equally plausible image is penalised as heavily as one that lands on mush. The qualitative artifacts show precisely this: many drafted images remain recognisable while differing compositionally from the baseline, whereas the skip arm’s 8.9 dB correctly identifies noise. SSIM adds structural sensitivity but inherits the same reference dependence. A perceptual or preference-based metric is needed, and a visual-ranking model — for instance the Gemma model already resident in the serving stack, used as a pairwise judge — is one possible instrument. We have not run this, and we decline to report an aesthetic-quality ranking we have not measured.

Retrieval is evaluated as a forecast, not yet as a live cache. Table 4 measures leakage-free held-out trajectory error, Table 5 measures a resident CPU kernel, and Figure 1 decodes six held-out endpoint forecasts. A live deployment additionally pays vector-index lookup, trajectory prefetch, GPU transfer, and cache churn. Those costs require the serving benchmark described in Section 7.

The aligned schedule is an offline true-anchor proxy. Table 3 resets every interval to a latent from the recorded teacher trajectory. A real branched sampler feeds an approximate teleported state into its next denoiser call, so errors can compound or be damped in ways this table cannot represent. The 30-step/four-call target, active-lane batch compaction, pairwise diversity, and decoded images from calibrated momentum remain queued production experiments. We do not convert the proxy improvement into an image-quality or latency claim. Because the stored 16-step callbacks begin after the first denoiser call, the current deployment table also lacks a coefficient for the first interval; the implementation explicitly falls back to unit-scale Taylor there.

No exact verification for unseen-prompt forecasts. The Taylor, walker, calibrated, and retrieved latents are not guaranteed to match the target model’s. Each trades quality for time, and a confidence gate bounds that trade heuristically rather than provably. Exact prompt-latent replay is the narrow exception because it resumes an already-computed deterministic request; it does not generalise to a new key and should not be counted as speculative verification.

10 Conclusion

Latent teleportation is a family of approximate trajectory forecasts and warm starts for diffusion, not an exact verifier. The premise is supported: the trajectory is smooth enough that one schedule-calibrated momentum scalar reduces held-out forecast error by 20–37% over raw Taylor through $k = 8$. Aligning a four-interval proxy reduces local error by 35.3%, and four distinct

near-optimal schedules stay within 2.4% of the best. A motion-pruned eight-neighbour residual further reduces held-out error by 3.3% and improves the six-prompt long-warp decode from 21.48 to 21.84 dB PSNR and from 0.773 to 0.788 SSIM. Variance shrinkage reaches 0.7883 SSIM, while the focused cache adapter improves the scalar gate at every horizon and reaches 21.92 dB. These results support a specific production architecture: preserve local momentum, retrieve only a small residual, align the anchors, and compact the batch lanes that need real refinement. The evaluated drafting schemes remove up to $3.2\times$ denoiser calls. All 36 learned and Taylor candidates in the image sweep preserve the requested subject or scene, while larger horizons introduce visible retention costs that the grids make explicit.

At the tested $512^2/16$ -step serving point, fixed text and VAE work make denoiser-only optimization the informative speed metric; the measured end-to-end speculative result is only $1.04\times$. The production hierarchy is therefore narrower than the mechanism menu: exact bounded replay for repeated keys; Taylor $k = 1$ for an explicitly approximate unseen-prompt mode; Q4 for shared-GPU headroom but Q8 for speed when it fits; and fused epilogues only after full-image parity. The exact tier measured $1.491\times$ on its fresh-key three-case paired run and survived the multi-hit stability soak, while the $1.74\times$ residual-RMS number remains an isolated kernel result.

The next research milestone is a held-out 30-step teacher experiment with approximately four real calls, decoded-image and preference evaluation, phase-separated timing, and peak-memory/OOM reporting. A batch-of-four run should then test staggered schedules with active-lane compaction and pairwise diversity. Confidence gating, live retrieved-residual prefetch, pruned students, and LoRA-space transport remain follow-on hypotheses rather than defaults.

Reproducibility

Code, the analysis harnesses, generated contact sheets, their source hashes, and the raw result JSON are in [Penkman \[2026a\]](#). Full-resolution source panels remain in the experiment store; the committed grids contain every panel used in the paper. The serving gateway is in [Penkman \[2026b\]](#). The LoRA adapter library is not open source; its integration mechanism is described in Section 3.5.

A Complete Image Hyperparameter Sweep

The image experiment is a full factorial sweep over forecaster family (learned walker, Taylor momentum, or destructive skip control), draft length $k \in \{1, 2, 3\}$, and six prompt slices, with one fixed seed. The three draft lengths correspond to 9, 6, and 5 real denoiser calls. Every non-reference cell is labelled with PSNR and SSIM against the same-seed 16-call output. The landscape sheet below contains all 54 candidate images and the six references; no cell is selected or omitted after viewing the output.

Full method $\times$ draft length $\times$ prompt sweep (seed 7)

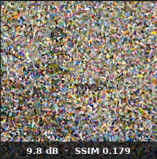
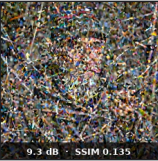
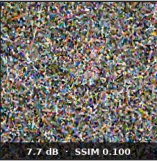
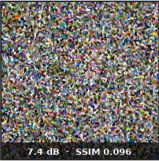
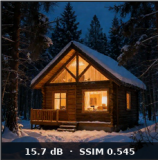
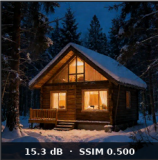
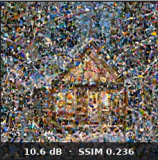
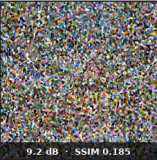
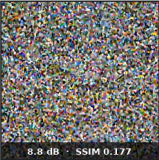
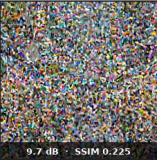
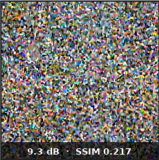
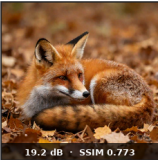
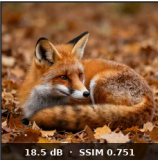
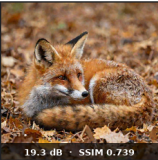
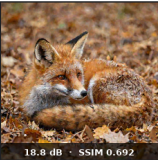
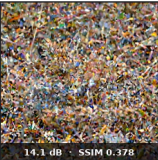
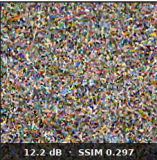
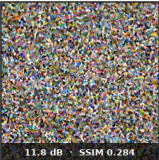
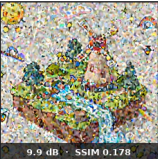
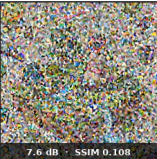
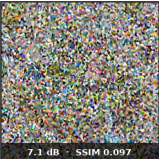
	Reference 16 real calls	Learned k=1	Learned k=2	Learned k=3	Taylor k=1	Taylor k=2	Taylor k=3	Skip control k=1	Skip control k=2	Skip control k=3
Golden-hour landscape a lighthouse on a cliff at golden hour, crashing waves	 reference	 14.4 dB · SSIM 0.732	 13.9 dB · SSIM 0.713	 13.9 dB · SSIM 0.713	 14.6 dB · SSIM 0.724	 14.2 dB · SSIM 0.682	 14.1 dB · SSIM 0.666	 11.6 dB · SSIM 0.260	 9.8 dB · SSIM 0.179	 9.3 dB · SSIM 0.165
Dramatic portrait portrait of an old fisherman with a pipe, dramatic lighting	 reference	 19.6 dB · SSIM 0.530	 20.3 dB · SSIM 0.565	 19.1 dB · SSIM 0.513	 20.9 dB · SSIM 0.681	 18.8 dB · SSIM 0.521	 17.7 dB · SSIM 0.453	 9.3 dB · SSIM 0.135	 7.7 dB · SSIM 0.100	 7.4 dB · SSIM 0.096
Snowy night scene a cozy cabin in a snowy forest at night, warm windows	 reference	 17.6 dB · SSIM 0.621	 15.7 dB · SSIM 0.545	 15.3 dB · SSIM 0.500	 18.6 dB · SSIM 0.705	 16.0 dB · SSIM 0.523	 14.7 dB · SSIM 0.454	 10.6 dB · SSIM 0.236	 9.2 dB · SSIM 0.185	 8.8 dB · SSIM 0.177
Cyberpunk / neon cyberpunk street market in the rain, neon signs	 reference	 17.4 dB · SSIM 0.672	 15.8 dB · SSIM 0.563	 14.7 dB · SSIM 0.504	 22.3 dB · SSIM 0.818	 17.2 dB · SSIM 0.583	 15.7 dB · SSIM 0.506	 11.2 dB · SSIM 0.279	 9.7 dB · SSIM 0.225	 9.3 dB · SSIM 0.217
Soft-light wildlife a fox curled up on autumn leaves, soft light	 reference	 20.1 dB · SSIM 0.806	 19.2 dB · SSIM 0.773	 18.5 dB · SSIM 0.751	 22.4 dB · SSIM 0.859	 19.3 dB · SSIM 0.739	 18.8 dB · SSIM 0.692	 14.1 dB · SSIM 0.378	 12.2 dB · SSIM 0.297	 11.8 dB · SSIM 0.284
Isometric illustration isometric tiny island with a waterfall and windmill	 reference	 19.8 dB · SSIM 0.835	 18.8 dB · SSIM 0.815	 17.6 dB · SSIM 0.791	 19.9 dB · SSIM 0.835	 18.3 dB · SSIM 0.774	 17.3 dB · SSIM 0.738	 9.9 dB · SSIM 0.178	 7.6 dB · SSIM 0.108	 7.1 dB · SSIM 0.097

Figure 3: Complete method $\times$ draft-length $\times$ prompt sweep. Columns vary method and k ; rows vary prompt content. This is the exhaustive visual counterpart to Tables 13 and 14.

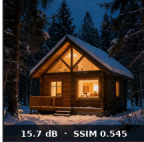
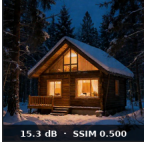
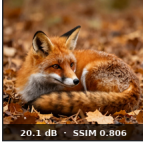
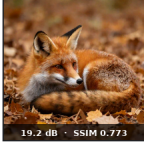
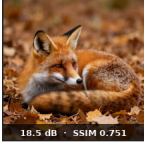
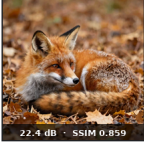
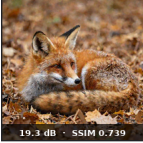
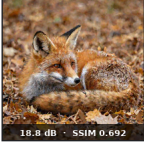
Draft-length hyperparameter ablation (seed 7)							
	Baseline 16 calls	Learned k=1	Learned k=2	Learned k=3	Taylor k=1	Taylor k=2	Taylor k=3
Golden-hour landscape a lighthouse on a cliff at golden hour, crashing waves							
	reference	14.4 dB · SSIM 0.732	13.9 dB · SSIM 0.713	13.9 dB · SSIM 0.713	14.6 dB · SSIM 0.724	14.2 dB · SSIM 0.682	14.1 dB · SSIM 0.666
Dramatic portrait portrait of an old fisherman with a pipe, dramatic lighting							
	reference	19.6 dB · SSIM 0.530	20.3 dB · SSIM 0.565	19.1 dB · SSIM 0.513	20.9 dB · SSIM 0.681	18.8 dB · SSIM 0.521	17.7 dB · SSIM 0.453
Snowy night scene a cozy cabin in a snowy forest at night, warm windows							
	reference	17.6 dB · SSIM 0.621	15.7 dB · SSIM 0.545	15.3 dB · SSIM 0.500	18.6 dB · SSIM 0.705	16.0 dB · SSIM 0.523	14.7 dB · SSIM 0.454
Cyberpunk / neon cyberpunk street market in the rain, neon signs							
	reference	17.4 dB · SSIM 0.672	15.8 dB · SSIM 0.563	14.7 dB · SSIM 0.504	22.3 dB · SSIM 0.818	17.2 dB · SSIM 0.583	15.7 dB · SSIM 0.506
Soft-light wildlife a fox curled up on autumn leaves, soft light							
	reference	20.1 dB · SSIM 0.806	19.2 dB · SSIM 0.773	18.5 dB · SSIM 0.751	22.4 dB · SSIM 0.859	19.3 dB · SSIM 0.739	18.8 dB · SSIM 0.692
Isometric illustration isometric tiny island with a waterfall and windmill							
	reference	19.8 dB · SSIM 0.835	18.8 dB · SSIM 0.815	17.6 dB · SSIM 0.791	19.9 dB · SSIM 0.835	18.3 dB · SSIM 0.774	17.3 dB · SSIM 0.738

Figure 4: Learned and Taylor draft-length slices at larger print scale. Increasing k removes more real denoiser calls and increases texture, geometry, and layout drift.

Skip-control draft-length ablation (seed 7)

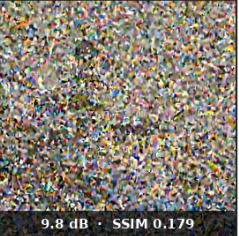
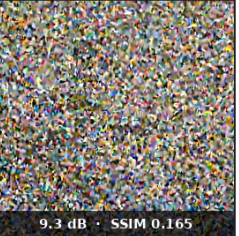
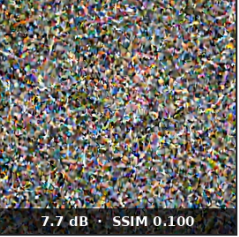
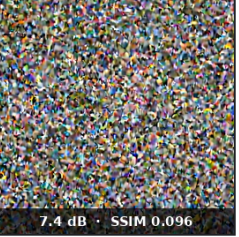
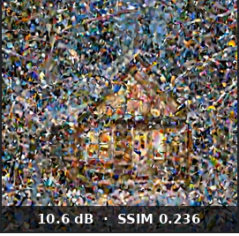
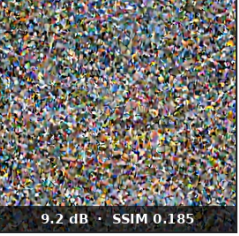
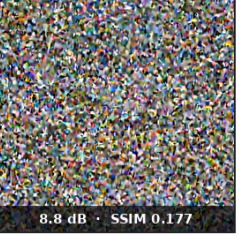
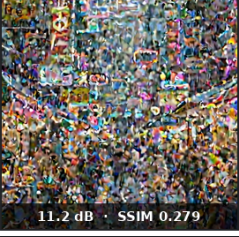
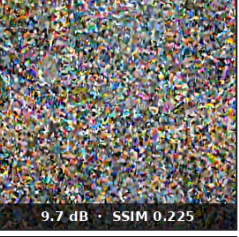
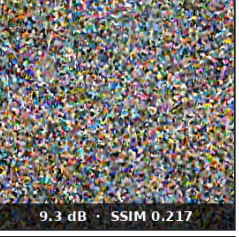
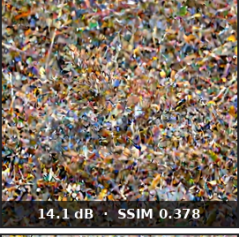
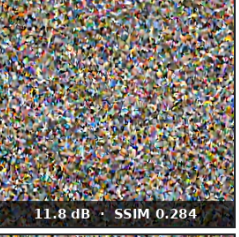
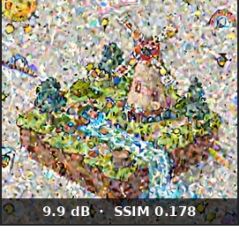
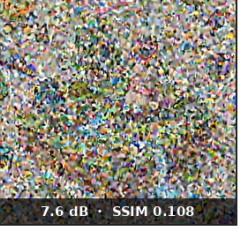
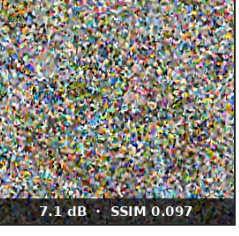
	Baseline 16 calls	Skip control k=1	Skip control k=2	Skip control k=3
Golden-hour landscape a lighthouse on a cliff at golden hour, crashing waves	 reference	 11.6 dB · SSIM 0.260	 9.8 dB · SSIM 0.179	 9.3 dB · SSIM 0.165
Dramatic portrait portrait of an old fisherman with a pipe, dramatic lighting	 reference	 9.3 dB · SSIM 0.135	 7.7 dB · SSIM 0.100	 7.4 dB · SSIM 0.096
Snowy night scene a cozy cabin in a snowy forest at night, warm windows	 reference	 10.6 dB · SSIM 0.236	 9.2 dB · SSIM 0.185	 8.8 dB · SSIM 0.177
Cyberpunk / neon cyberpunk street market in the rain, neon signs	 reference	 11.2 dB · SSIM 0.279	 9.7 dB · SSIM 0.225	 9.3 dB · SSIM 0.217
Soft-light wildlife a fox curled up on autumn leaves, soft light	 reference	 14.1 dB · SSIM 0.378	 12.2 dB · SSIM 0.297	 11.8 dB · SSIM 0.284
Isometric illustration isometric tiny island with a waterfall and windmill	 reference	 9.9 dB · SSIM 0.178	 7.6 dB · SSIM 0.108	 7.1 dB · SSIM 0.097

Figure 5: Destructive skip-control slice at larger print scale. The control uses the same call schedules but supplies no forecasted movement; structure collapses across every prompt slice.

B Two-Dimensional Retrieval Sweep

Table 17 expands the aggregate retrieval result into its two tuned dimensions. Rows increase forecast horizon and columns increase the number of motion-pruned neighbours. The comparison uses held-out trajectories in every cell. Retrieval helps at every horizon, and the gain grows with horizon and neighbourhood size; at horizon 8 the eight-neighbour residual lowers relL2 from 0.333 to 0.315.

horizon	calibrated local	1 neighbour	2 neighbours	4 neighbours	8 neighbours
1	0.176	0.175	0.174	0.173	0.173
2	0.203	0.201	0.200	0.199	0.198
4	0.251	0.247	0.245	0.244	0.243
8	0.333	0.323	0.320	0.317	0.315

Table 17: Held-out gated-residual sweep by forecast horizon and neighbour count. Cells are five-fold mean relL2; lower is better.

C Experiment and Artifact Inventory

experiment	evaluated units	sweep dimensions and artifacts
Trajectory forecast	200 trajectories; 375 held-out (t, k) cells	Eight horizons, five predictor families, five folds.
Anchor placement	200 trajectories; 15 fold/budget searches	3, 4, and 6 intervals; uniform, aligned, and four staggered schedules.
Trajectory retrieval	200 trajectories; 185 held-out step/horizon cells	Four horizons, four neighbour counts, direct-transport and residual architectures.
Cache adapter	160 train and 40 test trajectories	Four horizons; local, scalar gate, variance shrinkage, and 7,312-parameter adapter.
Decoded cache forecast	Six held-out prompts; five procedures	Endpoint reference plus four forecast procedures, with PSNR and SSIM.
End-to-end images	Six prompts; 72 saved PNGs	Three draft lengths, four arms, per-panel hashes, PSNR, and SSIM.
Steady-state timing	Five prompts per k ; three draft lengths	Denoisers calls, end-to-end speedup, and PSNR.
Native exact replay	Five resume steps, three fresh-key promotion cases, and a four-hit soak	Unsplit reference, cold prime, exact replay hashes, latency, cache metadata, admission retries, and post-request memory samples.
Native systems	Three quants, five step-cache points, one fused epilogue	Transformer iteration time, VRAM, failure status, PSNR, and sm_120 kernel/reference timing.

Table 18: Inventory of the experimental evidence used in the paper. Counts refer to evaluated units, not augmentation or repeated rendering of the same panel.

The generated tables and numeric macros are reconstructed by `scripts/paper_analysis.py`. Image sheets and the per-panel SHA-256 manifest are reconstructed by `scripts/paper_ablation_grids.py`. Raw summaries live under `results/speculative`; the 200 float16 trajectories and full-resolution PNG source panels remain in the experiment store because of their size.

References

Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwei Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint*, 2023.
- Xiaoliu Guan, Lielin Jiang, Hanqi Chen, Xu Zhang, Jiaying Yan, Guanzhong Wang, Yi Liu, Zetao Zhang, and Yu Wu. Forecasting when to forecast: Accelerating diffusion models with confidence-gated taylor. *arXiv preprint*, 2025.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. In *IEEE Transactions on Big Data*, 2019.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint*, 2024.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It is time to cache for video diffusion model. *arXiv preprint*, 2024a.
- Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Jiahui Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. *arXiv preprint*, 2025.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.
- Xingchao Liu, Xiwen Zhang, Jianzhu Ma, Jian Peng, and Qiang Liu. InstaFlow: One step is enough for high-quality diffusion-based text-to-image generation. In *International Conference on Learning Representations (ICLR)*, 2024b.
- Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint*, 2023.
- Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating diffusion models for free. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Lee Penkman. CuteDSL: Accelerated model inference via custom CuTeDSL and CUDA kernels. <https://github.com/lee101/cutedsl>, 2026a.
- Lee Penkman. omniserive-native: A native c inference gateway for a single GPU. Apache-2.0 licensed source release, 2026b.

- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align your steps: Optimizing sampling schedules in diffusion models. In *International Conference on Machine Learning (ICML)*, 2024.
- Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. *arXiv preprint*, 2023.
- Apoorv Saxena. Prompt lookup decoding. <https://github.com/apoorvumang/prompt-lookup-decoding>, 2023.
- Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. FORA: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint*, 2024.
- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning (ICML)*, 2023.
- Tongyi Lab, Alibaba Group. Z-image: An efficient foundation model for image generation. Model card and release notes, 2025.